

HANDWRITTEN NOTES

DIGITAL ELECTRONICS

Scanned Student Classroom Notes & Worksheets (Units 1 to 5)

Authentic CamScanner Handwritten Document Sections:

- Unit 1: Number Systems, Radix, Conversions & Boolean Theorems
- Unit 2: K-Maps, Red Looping Groups, Quine-McCluskey, TTL & CMOS
- Unit 3: Sequential Circuits, Latches, Clocked SR/JK/D/T, Counters
- Unit 4: Adders, Look-Ahead Carry, Comparators, MUX, Demux, Encoders
- Unit 5: Memories (SRAM, DRAM), EPROM UV Erasing, PROM, PLA, PAL

Includes actual handwritten pages scanned directly with CamScanner

Featuring genuine blue and red pen handwriting, hand-drawn logic diagrams, and K-map loops.

Digital Electronics E-Learning Platform

Digital Electronics Unit - 1

Number Systems

Number system is a basis for counting various items.

- Provides a set of symbols and rules for representing quantities.
- Helps in performing arithmetic operations in digital systems.

Example: $(18)_{10} = (10010)_2$

Radix Table

System	Radix (Base)	Symbols used
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal	16	0-9, A, B, C, D, E, F

1 0 1 1 0 1
↑ ↑
MSB LSB

Unit - II Digital Electronics : K-Map (Karnaugh Map)

- Karnaugh Map (K-Map): A graphical tool for simplifying Boolean expressions, to tool - simplization most variables - minimization symbols, and
- 2-Variable K-Map ($Y = f(A, B) = \sum m(1, 3)$)

A \ B	0	1
0	1	1
1		1

→ = B

- 3-Variable K-Map ($F(A, B, C) = \sum m(1, 3, 4, 6)$)

A \ BC	00	01	11	10
0	1	1		1
1		1	1	6

→ $A'C$
→ AC'

Result : $F = A'\bar{C} + \bar{A}C'$

- 4-Variable K-Map ($Z = f(W, X, Y, Z)$) for $\sum m(0, 1, 2, 3, 5, 7, 8, 9, 10, 11)$

WX \ YZ	00	01	11	10
00	m_0	m_1	m_2	m_3
01	0	1	1	m_5
11	1	1	1	1
10	1	m_8	m_9	m_{10}

→ $A' + A = \dots$
→ BD

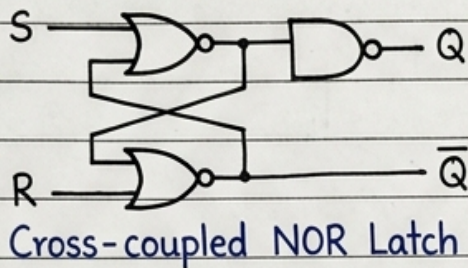
Result : $Z = X' + Z$

Unit - III Sequential Circuits & Flip-Flops

Flip-Flops is sequential flowtion with modificarons and inter-use of logic circuit.

Many form is showtown by sequentiality interportable mochi-norms.

3.1 SR Latch



Truth Table

Inputs	Clock	Next State
S R	0	Q_{t+1}
0 1	1	Q

Characteristic Equation:

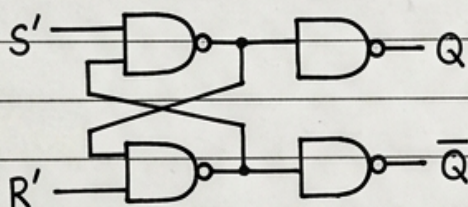
$$Q_{t+1} = S + R' \cdot Q_t \quad (R'S=0)$$

Truth Table

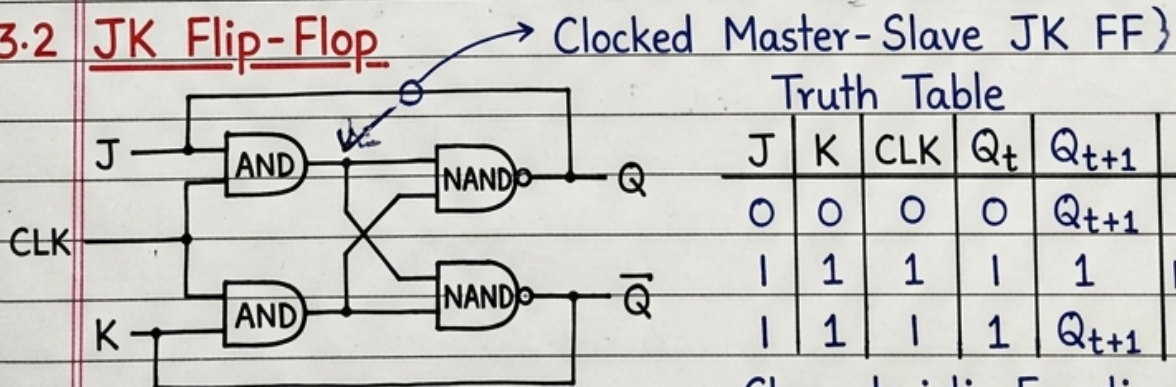
Inputs	S	R	Next State
S' R	0	1	Q_{t+1}
0 1	1	0	Q

Characteristic Equation:

$$Q_{t+1} = S + R' \cdot Q_t \quad (R'S=0)$$



3.2 JK Flip-Flop



Truth Table

J	K	CLK	Q_t	Q_{t+1}	Mode
0	0	0	0	Q_{t+1}	Pring
1	1	1	1	1	No-Lined.
1	1	1	1	Q_{t+1}	Proposed

Characteristic Equation:

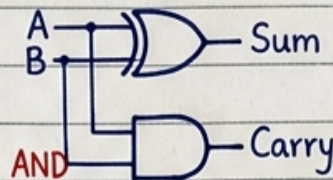
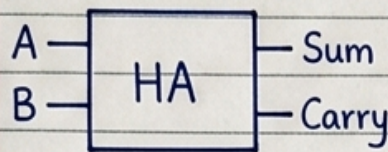
$$Q_{t+1} = J \cdot Q_t' + K' \cdot Q_t$$

Excitation Table

Q_t	Q_{t+1}	J	K
0	0	0	1
0	1	1	0
1	1	1	1

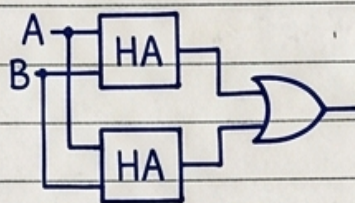
Combinational Circuits : Adders & Multiplexers

1. Half Adder (HA)



A	B	T	Sum	Carry
0	0	0	✓	0 ✓
0	1	1	1	2 ✓
1	1	0	0	1 ✓
4	1	1	✓	✓ Correct

2. Full Adder (FA)



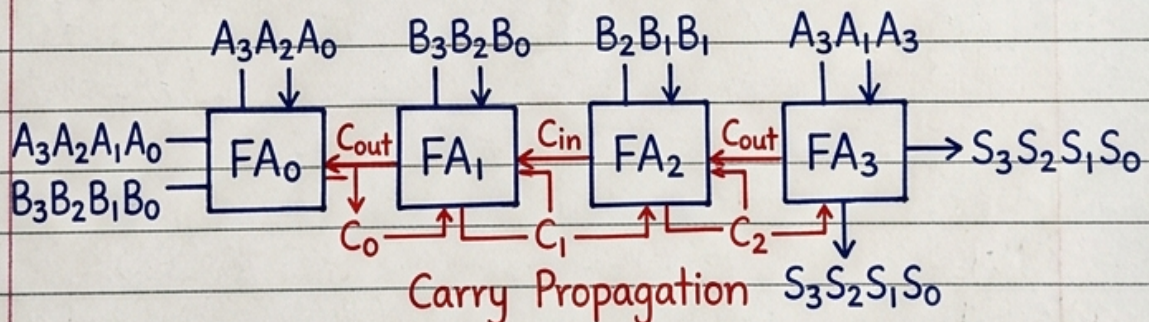
A	B	Cin	S	Cout
0	0	0	0	0 ✓
2	0	1	1	1 ✓
3	1	0	0	0 ✓
4	1	1	0	1 ✓
6	1	1	0	1 ✓
7	1	0	1	1 ✓

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = \overline{A}B + (A \oplus B)C_{in}$$

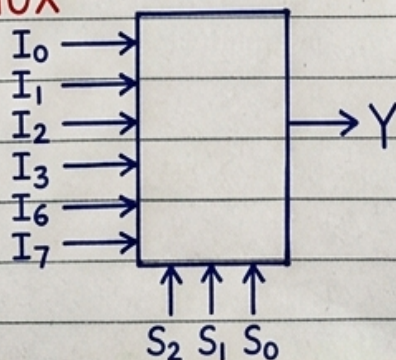
3. 4-bit Parallel Adder

✓ Correct



4. Multiplexers (MUX)

8x1 MUX

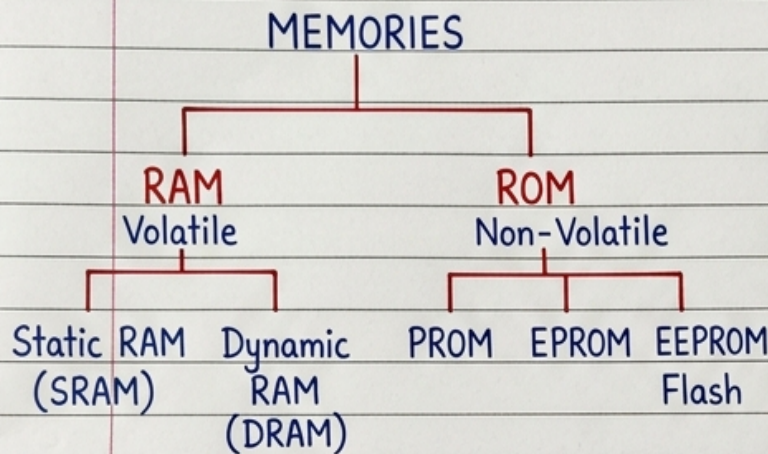


S2	S1	S0	Y
0	0	0	1 ✓
0	0	1	2 ✓
0	1	0	3 ✓
0	1	1	4 ✓
1	0	0	5 ✓
1	0	1	6 ✓
1	1	0	7 ✓
1	1	1	8 ✓

Unit - V Memories & PLDs

Memories are vitally of engineering student in in a main memories, starting volatile fuses → programmed & open connections.

Classification of Semiconductor Memories



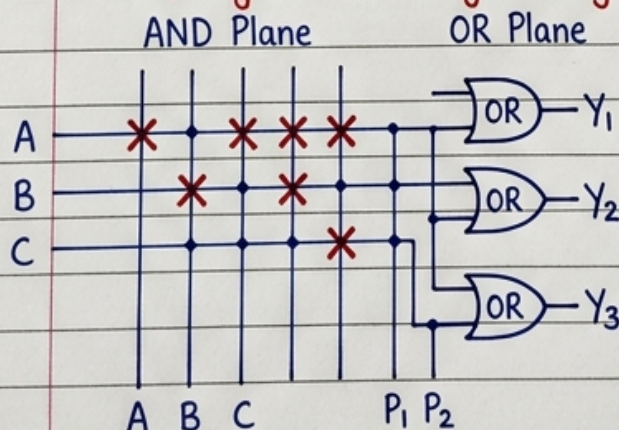
SRAM vs DRAM

Comparison Table

Parameter	SRAM	DRAM
Speed	Faster	Slower
Power	High	Low
Density	Low	High
Cost	High	Low
Mechanism	FF	Capacitor
Application		

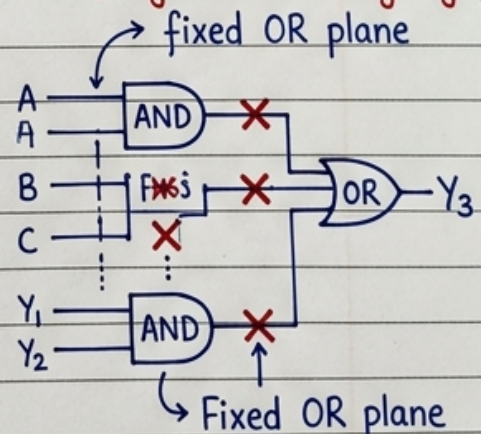
PLDs – the programmable logic array. PLDs as the vertical logic AND plane. PLDs are similar as fuses, know in the AND gates not are programmed or open connections.

PLA (Programmable Logic Array)



X = programmed • = open connection
 • = open connections

PAL (Programmable Array Logic)



Concept: Foundations of number representation, radix definition, and significance of bit weights.

> Number Systems Definition : -

- A number system is a basis for counting various items.
- Decimal number system has 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Modern computers communicate and operate with binary numbers which use only digits 0 & 1.
- When decimal quantities are represented in binary form, they take more digits.

> Radix (Base) : -

- The number of different symbols used by a number system is called Radix (or) Base.
- Binary: Radix = 2 (Symbols: 0, 1)
- Octal: Radix = 8 (Symbols: 0 to 7)
- Decimal: Radix = 10 (Symbols: 0 to 9)
- Hexadecimal: Radix = 16 (Symbols: 0 to 9, A, B, C, D, E, F)

> Most Significant Bit (MSB) : -

- The left-most bit which has greatest weight is called Most Significant Bit (MSB).
- Example in 10101: the leftmost 1 is MSB (weight $2^4 = 16$), rightmost 1 is LSB (weight $2^0 = 1$).

* Unit 1: Number Systems & Boolean Algebra

Positional Weights & Number System Structures

Page 2

Concept: Positional weighting of binary, octal, decimal, and hexadecimal systems.

> **Least Significant Bit (LSB) & Weights : -**

- The right-most bit which has least weight is called Least Significant Bit (LSB).
- Weight (or) Value represents the value of the digit position relative to the radix point.

> **Radix Weights Across Bases : -**

- Binary: ... $2^3, 2^2, 2^1, 2^0$. $2^{-1}, 2^{-2}, 2^{-3}$... Example: (10101.1101),
- Decimal: ... $10^3, 10^2, 10^1, 10^0$. $10^{-1}, 10^{-2}$... Example: (98.27)•€
- Octal: ... $8^3, 8^2, 8^1, 8^0$. $8^{-1}, 8^{-2}$... Example: (704.26)^
- Hexadecimal: ... $16^3, 16^2, 16^1, 16^0$. $16^{-1}, 16^{-2}$... Example: (4A.B2)•†

Concept: Technique of successive division by 2 for integer part and multiplication by 2 for fractional part.

> Conversion Rules :-

- Integer part: Successively divide by 2, collect remainders in upward direction (bottom to top).
- Fractional part: Successively multiply by 2, collect integer part of product downwards.

> Solved Examples :-

Q) Convert $(72)_{10}$ and $(0.95)_{10}$ to Binary

-> $72/2 = 36$ rem 0; $36/2 = 18$ rem 0; $18/2 = 9$ rem 0; $9/2 = 4$ rem 1; $4/2 = 2$ rem 0; $2/2 = 1$ rem 0; $1/2 = 0$ rem 1.

-> Reading upwards: $(72)_{10} = (1001000)_2$.

-> $0.95 \times 2 = 1.90$ (1); $0.90 \times 2 = 1.80$ (1); $0.80 \times 2 = 1.60$ (1); $0.60 \times 2 = 1.20$ (1); $0.20 \times 2 = 0.40$ (0); $0.40 \times 2 = 0.80$ (0); $0.80 \times 2 = 1.60$ (1)...

-> Reading downwards: $(0.95)_{10} = (0.11110011...)_{2}$.

* Unit 1: Number Systems & Boolean Algebra

Decimal to Octal & Hexadecimal Conversions

Page 4

Concept: Conversion of decimal numbers with fractional parts to octal (base 8) and hexadecimal (base 16).

> Decimal to Octal Examples :-

Q) Convert $(378.93)_{10}$ and $(35.45)_{10}$ to Octal

-> $378/8 = 47 \text{ rem } 2$; $47/8 = 5 \text{ rem } 7$; $5/8 = 0 \text{ rem } 5 \Rightarrow (378)_{10} = (572)_8$.

-> $0.93 \times 8 = 7.44 (7)$; $0.44 \times 8 = 3.52 (3)$; $0.52 \times 8 = 4.16 (4)$; $0.16 \times 8 = 1.28 (1)$...

-> $(378.93)_{10} = (572.7341217...)_{8}$.

-> $(35.45)_{10} = (43.34632...)_{8}$.

> Decimal to Hexadecimal Example :-

Q) Convert $(2598.675)_{10}$ to Hexadecimal

-> $2598/16 = 162 \text{ rem } 6$; $162/16 = 10 (A) \text{ rem } 2 \Rightarrow (2598)_{10} = (A66)_{16}$.

-> $0.675 \times 16 = 10.8 (A)$; $0.8 \times 16 = 12.8 (C)$; $0.8 \times 16 = 12.8 (C)$...

-> $(2598.675)_{10} = (A66.ACC...)_{16}$.

* Unit 1: Number Systems & Boolean Algebra

Binary, Octal, Hexadecimal to Decimal Conversion

Page 5

Concept: Direct expansion using positional polynomial powers.

> Binary to Decimal :-

Q) Convert (101011.11001), to Decimal

$$\rightarrow = 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4} + 1 \times 2^{-5}$$

$$\rightarrow = 32 + 8 + 2 + 1 + 0.5 + 0.25 + 0.03125 = (43.78125)_{10}$$

> Octal & Hexadecimal to Decimal :-

Q) Convert (742.2)₈ and (A26.B1)₁₆ to Decimal

$$\rightarrow (742.2)_8 = 7 \times 8^2 + 4 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = 448 + 32 + 2 + 0.25 = (482.25)_{10}$$

$$\rightarrow (A26.B1)_{16} = 10 \times 16^2 + 2 \times 16^1 + 6 \times 16^0 + 11 \times 16^{-1} + 1 \times 16^{-2} = 2560 + 32 + 6 + 0.6875 = (2598.6875)_{10}$$

* Unit 1: Number Systems & Boolean Algebra

Binary-Octal & Binary-Hexadecimal Shortcuts

Page 6

Concept: Rapid grouping methods: 3-bit groups for octal, 4-bit groups for hexadecimal.

> Binary to Octal & Vice-Versa :-

- Group binary bits in 3s starting from radix point (left for integer, right for fraction).
- $(11011011.0110111), \rightarrow (011)(011)(011).(011)(011)(100) \rightarrow (333.334)^{\wedge}.$
- Octal to Binary: replace each octal digit with exact 3-bit binary equivalent.

> Binary to Hexadecimal :-

- Group binary bits in 4s starting from radix point.
- $(11011011.01101110), \rightarrow (1101)(1011).(0110)(1110) \rightarrow (DB.6E)^{\dagger}.$
- $(001111000111.11110000), \rightarrow (0011)(1100)(0111).(1111)(0000) \rightarrow (3C7.F0)^{\dagger}.$

* Unit 1: Number Systems & Boolean Algebra

Octal-Hexadecimal Conversions via Binary Intermediate

Page 7

Concept: Two-step conversion process linking octal and hexadecimal using binary as the bridge.

> Hexadecimal to Binary :-

Q) Convert (A26.B1)₁₆ and (1BCD)₁₆ to Binary

-> (A26.B1)₁₆ = (1010 0010 0110 . 1011 0001)₂

-> (1BCD)₁₆ = (0001 1011 1100 1101)₂

> Octal to Hexadecimal :-

Q) Convert (746.22)₈ and (256.235)₈ to Hexadecimal

-> Step 1: (746.22)₈ = (111 100 110 . 010 010)₂

-> Step 2: Group into 4s: (0001 1110 0110 . 0100 1000)₂ = (1E6.48)₁₆

-> (256.235)₈ = (010 101 110 . 010 011 101)₂ -> (0AE.4E8)₁₆

Complete 0-15 Multi-Base Conversion Reference Table

Concept: Comprehensive cross-reference table comparing Decimal, Binary, Octal, and Hexadecimal.

> Reference Master Table (0 to 15) :-

Decimal	Binary (8421)	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

* Unit 1: Number Systems & Boolean Algebra

Digital Logic Gates: Basic Gates (NOT, AND)

Page 9

Concept: Definitions, ANSI logic symbols, transfer functions, and truth tables for NOT and AND gates.

> **Classification of Logic Gates** :-

- Basic Gates: NOT, AND, OR
- Universal Gates: NAND, NOR
- Special Gates: Ex-OR, Ex-NOR

> **NOT Gate (Inverter)** :-

- Definition: Produces complement of input.
- Truth table: $A=0 \rightarrow Y=1$; $A=1 \rightarrow Y=0$.

$$\text{Eq : } Y = A \text{ (o r } A' \text{)}$$

> **AND Gate** :-

- Produces HIGH output only when ALL inputs are HIGH. If at least one input is LOW, output is LOW.
- Truth table: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$, $(1,1) \rightarrow 1$.

$$\text{Eq: } Y = A \cdot B$$

Concept: Definitions, ANSI logic symbols, Boolean transfer equations, and truth tables.

> OR Gate : -

- Produces HIGH output when AT LEAST ONE input is HIGH. Low only when all inputs are low.
- Truth table: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->1.

$$\text{Eq: } Y = A + B$$

> NAND Gate (Universal) : -

- Equivalent to AND gate followed by NOT gate.
- Output is LOW only when all inputs are HIGH; HIGH if at least one input is LOW.
- Truth table: (0,0)->1, (0,1)->1, (1,0)->1, (1,1)->0.

$$\text{Eq : } Y = (A \cdot B)$$

> NOR Gate (Universal) : -

- Equivalent to OR gate followed by NOT gate.
- Output is HIGH only when all inputs are LOW; LOW if at least one input is HIGH.
- Truth table: (0,0)->1, (0,1)->0, (1,0)->0, (1,1)->0.

$$\text{Eq : } Y = (A + B)$$

Concept: Exclusive-OR and Exclusive-NOR gate equations, symbols, parity behavior, and truth tables.

> Exclusive-OR (XOR) Gate :-

- Produces HIGH output when inputs are DIFFERENT (odd parity of 1s).
- Output is LOW when both inputs are identical.
- Truth table: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0.

$$\text{Eq : } Y = A \oplus B = A \bar{B} + \bar{A} B$$

> Exclusive-NOR (XNOR) Gate / Equivalence Gate :-

- Produces HIGH output when both inputs are SAME (coincidence/equality detector).
- Truth table: (0,0)->1, (0,1)->0, (1,0)->0, (1,1)->1.

$$\text{Eq : } Y = A \odot B = (A \oplus B) = A \bar{B} + \bar{A} B$$

Boolean Algebra Laws: Identity, Commutative, Complementation

Concept: Axioms, identity elements, commutative laws with truth table proofs, and double complementation.

> Boolean Algebra Concept :-

- Mathematical system defining logical operations (AND, OR, NOT) performed on variables (A, B, C...).
- Boolean functions: expressions constructed by connecting Boolean constants/variables with operators. Ex:
 $f(A, B, C) = (A + B)C$.

> Identity & Commutative Laws :-

- Commutative law is verified by truth table: column (A+B) matches (B+A) exactly for all four binary combinations.

Eq: Identity: $A \cdot 1 = A$, $A + 0 = A$, $A + 1 = 1$, $A \cdot 0 = 0$

Eq: Commutative: $A + B = B + A$, $A \cdot B = B \cdot A$

Eq: Double Complementation: $A = A$

* Unit 1: Number Systems & Boolean Algebra

Distributive & Associative Properties with Proofs

Page 13

Concept: Truth table proof of Distributive laws (including $A + BC = (A+B)(A+C)$) and Associative laws.

> **Distributive Property : -**

- Proof of $A + BC = (A+B)(A+C)$ via 8-row truth table demonstrates column $A+BC$ is identical to $(A+B)(A+C)$ across all 8 input combinations.

$$\text{Eq: 1) } A(B + C) = AB + AC$$

$$\text{Eq: 2) } A + BC = (A + B)(A + C) \text{ [Crucial Boolean dual identity]}$$

> **Associative Property : -**

$$\text{Eq: 1) } A + (B + C) = (A + B) + C$$

$$\text{Eq: 2) } A \cdot (BC) = (AB) \cdot C$$

Absorption, Idempotency & Redundant (Elimination) Laws

Concept: Complementarity, sameness (idempotency), absorption identities and proofs, and redundant rule.

> Complementarity & Idempotency :-

$$\text{Eq : } A \cdot \bar{A} = 0, \quad A + \bar{A} = 1$$

$$\text{Eq : } A + A = A, \quad A \cdot A = A$$

> Absorption Law :-

$$\text{Eq : } A + AB = A \quad [\text{Proof : } A(1 + B) = A \cdot 1 = A]$$

$$\text{Eq : } A(A + B) = A \quad [\text{Proof : } A \cdot A + AB = A + AB = A(1 + B) = A]$$

> Redundant (Elimination) Law :-

$$\text{Eq : } A + A \cdot B = A + B \quad [\text{Proof : } (A + A \cdot B)(A + B) = A + B]$$

$$\text{Eq : } A(A + B) = AB \quad [\text{Proof : } AA + AB = 0 + AB = AB]$$

* Unit 1: Number Systems & Boolean Algebra

De Morgan's Theorems & Duality Principle

Page 15

Concept: Two De Morgan theorems with truth table proofs, and duality theorem rules and examples.

> De Morgan's Theorems :-

- Both proven by 4-row truth tables matching columns exactly.

Eq : Theorem 1 : $(A \cdot B) = \overline{A + B}$ ("Complement of A and B is equal to the sum of the complements of A and B")
Eq : Theorem 2 : $(A + B) = \overline{A \cdot B}$ ("Complement of the sum of A and B is equal to the product of the complements of A and B")

> Duality Theorem Rules :-

1. Change each OR sign (+) to AND ($\cdot$).
 2. Change each AND sign ($\cdot$) to OR (+).
 3. Complement any 0 or 1 appearing in expression ($0 \leftrightarrow 1$).
- NOTE: Keep variables as they are (do NOT complement variables for dual!).

Q) Find Dual of: $A + A = 1$; $x + (x \cdot y + x \cdot z)$; $ABC + DE + BCE$

-> Dual of $A + A = 1$ is $A \cdot A = 0$.

-> Dual of $x + (x \cdot y + x \cdot z)$ is $x \cdot [(x + y) \cdot (x + z)]$.

-> Dual of $ABC + DE + BCE$ is $(A + B + C)(D + E)(B + C + E)$.

Concept: Exam-standard Boolean minimization and complement problems solved step-by-step.

> **Problem 1: Complement of $ABC + DE + BCE$:-**

$$\bullet (ABC + DE + BCE)' = (ABC)' \cdot (DE)' \cdot (BCE)' = (A' + B' + C')(D' + E')(B + C'E)$$

> **Problem 2: Minimize $f = A'B + A'BC' + A'BCD + A'BC'D'E$:-**

- Factor out A'B: $f = A'B [1 + C' + CD + C'D'E]$.
- Since $1 + \text{anything} = 1$: $f = A'B(1) = A'B$.
- Complement $f' = (A'B)' = A + B'$.

> **Problem 3: Reduce $f = A[B + C((AB + AC))]$:-**

- Applying De Morgan to inner term: $(AB + AC)' = (AB)' \cdot (AC)' = (A' + B')(A' + C)$
- $f = A[B + C(A' + B'C)] = A[B + A'C + 0] = AB + AA'C = AB + 0 = AB$

* Unit 1: Number Systems & Boolean Algebra

Binary Arithmetic & Signed Binary Number Systems

Page 17

Concept: Binary addition, binary subtraction truth tables, Sign-Magnitude, 1s complement, and 2s complement.

> Binary Addition & Subtraction :-

- Addition: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$ (carry 1). Ex: $(23)_{10} + (15)_{10} = (38)_{10}$
- Subtraction: $0-0=0$, $1-0=1$, $1-1=0$, $0-1=1$ (borrow 1). Ex: $(11)_{10} - (5)_{10} = (6)_{10}$

> Representation of Signed Numbers :-

- 1. Sign-Magnitude: MSB=0 for positive, MSB=1 for negative, remaining bits are magnitude. $+7 = 0111$, $-7 = 1111$.
- 2. 1s Complement: Invert each bit ($1 \rightarrow 0$, $0 \rightarrow 1$). $+7 = 0111$, $-7 = 1000$.
- 3. 2s Complement: Add 1 to LSB of 1s complement. $+7 = 0111$, $-7 = 1000 + 1 = 1001$.

Concept: Hardware advantages of complement arithmetic and detailed 1s complement subtraction cases.

> Advantages of Complement Subtraction : -

- Hardware reduction: instead of having separate digital circuits for addition & subtraction, only addition circuits perform both operations.

> 1's Complement Subtraction Procedure : -

- Step 1: Keep the first number (minuend) as it is.
- Step 2: Find the 1's complement of the 2nd number (subtrahend).
- Step 3: Add Step 1 and Step 2.
- Step 4 Case 1: If carry is present, result is positive. Add the end-around carry to LSB to get final result.
- Step 4 Case 2: If carry is NOT present, result is negative. Take 1's complement of the result.

Q) Subtract 48 - 23 using 1's complement (8-bit)

-> $48 = 00110000$; $23 = 00010111$.

-> 1's comp of 23 = 11101000 .

-> Add: $00110000 + 11101000 = (1) 00011000$ (carry 1).

-> Carry present -> Add carry to LSB: $00011000 + 1 = 00011001 = (+25)_{10}$.

Concept: Detailed step-by-step algorithms and solved examples for 2s complement subtraction.

> Negative Result in 1s Complement Example : -

- $23 - 48$: $23 = 00010111$; 1s comp of $48 = 11001111$.
- Sum = 11100110 . No carry $\rightarrow$ result is negative.
- 1s comp of $11100110 = 00011001 = -(25)_{10}$.

> 2's Complement Subtraction Procedure : -

- Step 1: Keep 1st number as it is.
- Step 2: Find 2's complement of 2nd number.
- Step 3: Add Step 1 and Step 2.
- Step 4 Case 1: If carry is present, result is positive. DISCARD (IGNORE) the carry!
- Step 4 Case 2: If carry is NOT present, result is negative. Take 2's complement of the sum to get magnitude.

Q) Subtract 25 - 14 using 2's complement

- $\rightarrow 25 = 00011001$; $14 = 00001110$.
- $\rightarrow$ 2's comp of $14 = 11110001 + 1 = 11110010$.
- $\rightarrow$ Add: $00011001 + 11110010 = (1) 00001011$.
- $\rightarrow$ Discard carry $\rightarrow$ Result is positive = $(+11)_{10} = (00001011)_{2}$.

Concept: Taxonomy of Numeric and Alphanumeric codes: weighted, non-weighted, cyclic, error codes.

> Binary Codes Tree : -

- Numeric Codes: Weighted (8421, 2421, 5421, 6311) vs Non-Weighted (Gray, Excess-3).
- Self-Complementing Codes: 2421, Excess-3, Gray.
- Reflective Codes: Excess-3, 2421.
- Cyclic (Unit Distance) Codes: Gray Code.
- Sequential Codes: 8421, Excess-3.
- Error Detecting & Correcting: Hamming Code.
- Alphanumeric Codes: ASCII, EBCDIC, Hollerith.

BCD (Binary Coded Decimal) & Weighted Codes Table

Concept: Detailed explanation of BCD and comparative table of 0-15 in 8421, 2421, and 6311 codes.

> BCD (Binary Coded Decimal) :-

- Weighted numeric code where each decimal digit (0-9) is represented as a group of 4 bits.
- Example: (12)₁₀ in binary is (1100)₂, but in BCD it is (0001 0010)_{BCD}.
- Valid BCD digits are 0000 to 1001. States 1010 to 1111 are INVALID BCD.

* Unit 1: Number Systems & Boolean Algebra

Excess-3 (XS-3) & Self-Complementing Codes

Page 22

Concept: Derivation of Excess-3 code by adding 3 (0011) and demonstration of self-complementing property.

> **Excess-3 (XS-3) Code : -**

- Modified form of BCD derived by adding 3 (0011), to each decimal digit.
- Example: (12)₁₀ in BCD = 0001 0010 $\rightarrow$ XS-3 = 0100 0101.

> **Self-Complementing Code Principle : -**

- A code is self-complementing if the 9's complement of N (i.e. $9 - N$) can be obtained directly by inverting all 0s to 1s and 1s to 0s.
- Proof in XS-3: For $N=7$ (1010), $9-7 = 2$ (0101). Notice 1010 inverted is 0101!
- For $N=4$ (0111), $9-4 = 5$ (1000). 0111 inverted is 1000! Hence XS-3 is self-complementing.

* Unit 1: Number Systems & Boolean Algebra

Gray Code (Cyclic / Unit Distance) Conversions

Page 23

Concept: XOR logic equations for Binary to Gray and Gray to Binary conversions with diagrams.

> Binary to Gray Code Conversion :-

E q : $G \bullet = B \bullet$ (M S B r e m a i n s s a m e)
E q : $G , = B \bullet \bullet B ,$
E q : $G f = B , \bullet B f$
E q : $G \text{''} = B f \bullet B \text{''}$

Q) Convert Binary 1101 to Gray Code

-> $G \bullet = 1$; $G , = 1 \bullet 1 = 0$; $G f = 1 \bullet 0 = 1$; $G \text{''} = 0 \bullet 1 = 1$ -> Gray = 1011.

> Gray to Binary Code Conversion :-

E q : $B \bullet = G \bullet$
E q : $B , = B \bullet \bullet G ,$
E q : $B f = B , \bullet G f$
E q : $B \text{''} = B f \bullet G \text{''}$

Binary-Gray 0-15 Table & Error Control Overview

Concept: Complete conversion table 0-15 and introduction to parity bit and Hamming code concepts.

> Binary to Gray Verification :-

- Observe that each consecutive Gray code differs by EXACTLY 1 bit (no switching glitches!).

* Unit 1: Number Systems & Boolean Algebra

Parity Bits & Hamming Code Formulation

Page 25

Concept: Parity bit generation and Hamming inequality $2^P \geq M + P + 1$ for single bit error correction.

> Parity Bit :-

- Extra bit included with message to make total count of 1s either even or odd.
- Even parity: appends bit so total 1s is even. Ex: for 101 (two 1s), parity bit = 0 $\rightarrow$ 0 101.
- Odd parity: appends bit so total 1s is odd. Ex: for 101, parity bit = 1 $\rightarrow$ 1 101.

> Hamming Code Formulation :-

- Parity bits located at power-of-2 positions: 1 (2^0), 2 (2^1), 4 (2^2), 8 (2^3)...
- P_1 checks positions: 1, 3, 5, 7, 9, 11...
- P_2 checks positions: 2, 3, 6, 7, 10, 11...
- P_4 checks positions: 4, 5, 6, 7, 12, 13, 14, 15...
- P_8 checks positions: 8, 9, 10, 11, 12, 13, 14, 15...

Eq : $2^P \geq M + P + 1$ (M = data bits , P = parity bits)

Concept: Full worked examples for 7-bit even parity and 9-bit odd parity Hamming encoded words.

> Encode Data 1011 into 7-bit Even Parity Hamming Code : -

- $M = 4$. $2^P \leq 4 + P + 1 \rightarrow P = 3$ parity bits (total length = 7 bits).
- Bit positions: $7=D_7=1$, $6=D_6=0$, $5=D_5=1$, $4=P_4$, $3=D_3=1$, $2=P_2$, $1=P_1$.
- P_1 checks (1,3,5,7) $\rightarrow P_1, 1, 1, 1$ (three 1s) $\rightarrow P_1 = 1$ (to make even).
- P_2 checks (2,3,6,7) $\rightarrow P_2, 1, 0, 1$ (two 1s) $\rightarrow P_2 = 0$.
- P_4 checks (4,5,6,7) $\rightarrow P_4, 1, 0, 1$ (two 1s) $\rightarrow P_4 = 0$.
- Encoded Message: [1 0 1 0 1 0 1].

> Encode Data 11101 using Odd Parity Hamming Code : -

- $M = 5 \rightarrow P = 4$ parity bits (total length = 9 bits).
- Encoded Message: [1 0 1 1 0 1 1 0 0].

* Unit 1: Number Systems & Boolean Algebra

Hamming Code Error Detection & Correction

Page 27

Concept: Determining error syndromes $C_3 C_2 C_1$ to pinpoint bit error position and invert corrupt bit.

> Detect & Correct Received Word: 0100011 (Even Parity) :-

- Bit positions 7 to 1: $b_7=0, b_6=1, b_5=0, b_4=0, b_3=0, b_2=1, b_1=1$.
- C_1 checks (1,3,5,7): $1 + 0 + 0 + 0 = 1$ (odd) $\rightarrow C_1 = 1$.
- C_2 checks (2,3,6,7): $1 + 0 + 1 + 0 = 2$ (even) $\rightarrow C_2 = 0$.
- C_3 checks (4,5,6,7): $0 + 0 + 1 + 0 = 1$ (odd) $\rightarrow C_3 = 1$.
- Error syndrome ($C_3 C_2 C_1$), $= (1 0 1)$, $= (5)_{10}$.
- Error is in 5th bit! Invert bit 5 from 0 to 1.
- Corrected Code: [0 1 1 0 0 1 1].

ASCII Code (American Standard Code for Information Interchange)

Concept: 7-bit alphanumeric code structure, hex mapping for characters A-Z, a-z, 0-9, and conversion examples.

> ASCII Structure :-

- 7-bit code representing $2^7 = 128$ characters.
- A to Z: (41) to (5A)
- a to z: (61) to (7A)
- 0 to 9: (30) to (39)

> Worked Examples :-

Q) Convert "Eee2" and "Binary" to ASCII binary strings

-> E -> 45 -> 1000101,; e -> 65 -> 1100101,; 2 -> 32 -> 0110010,.
-> "Binary": B(42)->1000010, i(69)->1101001, n(6E)->1101110, a(61)->1100001, r(72)->1110010, y(79)->1111001.

Concept: Systematic conversion of AND-OR-INVERT logic into pure NAND gates using bubble placement.

> Why NAND & NOR are Universal Gates :-

- Any basic logic gate (NOT, AND, OR) and any combinational logic circuit can be implemented exclusively using only NAND or only NOR gates.

> NAND-NAND Implementation Algorithm :-

- 1. Draw AOI (AND-OR-INVERT) logic for function.
- 2. Add bubble at output of AND gate and inputs of OR gate.
- 3. Wherever bubble is placed in circuit, place NOT gate in same path to compensate.
- 4. Eliminate double inversions.
- 5. Replace single input NOT gates with NAND gates.

> Example: XOR Implementation with NAND :-

- Standard conversion results in 5 NAND gates (or 4 minimal).

$$Eq : f = A B + A \bar{B} = A \cdot B$$

Concept: Conversion of logic functions into pure NOR gates, demonstrating 5 NOR gates for XOR and 6 NOR gates for XNOR.

> NOR-NOR Algorithm : -

- 1. Draw AOI logic.
- 2. Put bubbles at output of OR gate and input of AND gates.
- 3. Compensate with inverters.
- 4. Eliminate double inversions.
- 5. Replace inverters with single-input NOR gates.

* Unit 1: Number Systems & Boolean Algebra

Synthesizing All Basic Gates Using NAND Gates

Page 31

Concept: Schematic diagrams showing NOT (1), AND (2), and OR (3) using only 2-input NAND gates.

> Gate Realizations with NAND :-

- NOT gate: Tie both inputs of NAND together $\rightarrow Y = (A \cdot A) = \bar{A}$ (1 NAND).
- AND gate: NAND gate followed by NAND inverter $\rightarrow Y = ((A \cdot B) \cdot (A \cdot B)) = AB$ (2 NANDs).
- OR gate: Invert A and B with NAND inverters, feed to 3rd NAND $\rightarrow Y = (\bar{A} \cdot \bar{B}) = \overline{A \cdot B}$ (3 NANDs).

Universal Gates Master Gate Count Summary Table

Concept: Definitive competitive exam table comparing minimum NAND vs NOR gate counts for all 7 logic gates.

> Minimum Gate Counts : -

Target Gate	Only NAND Gates	Only NOR Gates
1. AND	2	3
2. OR	3	2
3. NOT	1	1
4. NAND	1	4
5. NOR	4	1
6. XOR	4 or 5	5
7. XNOR	5	4 or 5

Concept: Definitions of canonical Sum of Products (minterms m_i) and Product of Sums (maxterms M_i).

> Sum of Products (SOP) & Minterms :-

- A minterm (m_i) is a product term in which all variables exist either in normal or complement form.
- Minterm is 1 for corresponding input combination.

$$\text{Eq : } f(A, B, C) = \sum m(3, 5, 6, 7)$$

> Product of Sums (POS) & Maxterms :-

- A maxterm (M_i) is a sum term in which all variables exist either in normal or complement form.
- Maxterm is 0 for corresponding input combination.

$$\text{Eq : } f(A, B, C) = \prod M(0, 1, 2, 4)$$

Concept: Technique of multiplying missing variables by $(X + X)$ for SOP and adding

> Convert $f = A B + C + A B C$ to Canonical SOP : -

- $f = A B (C + C) + C (A + A) (B + B) + A B C$
- $= A B C + A B C + A B C + A B C + A B C + A B C + A B C$
- $= m_f + m_{,} + m_{\dagger} + m_{,,} + m_{\text{€}} + m_{\ddagger}$
- $f(A, B, C) = \sum m(0, 2, 3, 4, 6, 7)$.

> Convert to Standard POS : -

- Maxterms are remaining combinations: $f(A, B, C) = \prod M(1, 5)$.

Karnaugh Map (K-Map) Fundamentals & Cell Grouping Rules

Concept: Graphical minimization technique, Gray code coordinate labeling, and 2^n cell grouping rules.

> Concept of K-Map :-

- K-Map is a graphical method of simplifying Boolean expressions.
- Advantage: Systematic approach that avoids unpredictable algebraic intuition.
- Consists of 2^n cells for n variables: 2-var = 4 cells; 3-var = 8 cells; 4-var = 16 cells.

> Grouping Rules :-

- 16 1s (complete 4-var map) -> reduces 4 variables (result = 1).
- 8 1s (Octet) -> reduces 3 variables.
- 4 1s (Quad) -> reduces 2 variables.
- 2 1s (Pair) -> reduces 1 variable.
- Single 1 -> reduces 0 variables.

2-Variable & 3-Variable K-Map Solved Problems

Concept: Grouping pairs and quads across 3-variable maps with Gray code adjacency.

> Examples :-

- $f = \sum m(0,2,3) \rightarrow$ Group (0,2) gives B ; Group (2,3) gives A $\rightarrow f = B + A$.
- $f = \sum m(0,5,6,7) \rightarrow f = A B C + AC + AB$.
- $f = \sum m(0,2,4,6) \rightarrow$ All corners group into a quad $\rightarrow f = C$!

4-Variable K-Map Simplification & Gate Implementation

Concept: Minimizing 4-variable expressions and drawing AOI gate circuit implementations.

> **Example:** $f(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 7, 11, 15)$: -

- Quad (0, 1, 2, 3) gives $A \bar{B}$.
- Quad (3, 7, 11, 15) gives CD .
- Pair (0, 4) gives $A \bar{C} \bar{D}$.
- Final simplified expression: $f = A \bar{B} + CD + A \bar{C} \bar{D}$.

Prime Implicants, Don't Care Conditions & POS K-Maps

Concept: Definitions of Prime Implicant, Essential Prime Implicant, handling don't care terms (d), and POS maps.

> Prime Implicants & Don't Cares :-

- Prime Implicant: A group of 2• adjacent cells that cannot be combined into
- Essential Prime Implicant (EPI): A prime implicant that covers at least one '1' not covered by any other prime implicant.
- Don't Care conditions (d or X): Outputs that never occur in practice or whose output value doesn't matter. Can be grouped as 1 if helpful, or left as 0.

POS K-Map Reduction & Introduction to Quine-McCluskey

Concept: Grouping 0s in K-maps to obtain simplified Product of Sums (POS), and need for Tabular Method for ≥ 5 variables.

> POS Minimization Example :-

- $f(A, B, C) = \sum m(1, 4, 6, 7) \rightarrow$ Grouping 0s gives $f = (A + B)(B + C)(A + B + C)$

> Quine-McCluskey (Tabular) Method :-

- Map method is difficult when variables exceed 4 or 5 because 3D visualization is required.
- Tabular method is algorithmic and suitable for computer programming.

Quine-McCluskey Method: Grouping by Number of 1s

Concept: Grouping minterms by number of 1s, matching adjacent binary terms differing by 1 bit.

> Step-by-Step Procedure : -

- Step 1: List all minterms in binary form.
- Step 2: Partition into groups based on the count of 1s (Group 0, Group 1, Group 2, etc.).
- Step 3: Compare each term in group i with group $i+1$. If they differ in exactly one bit position, place a dash (-) in that position and place check mark (✓) on both terms.
- Step 4: Repeat for pairs and quads until no further combinations are possible.

* Unit 2: K-Maps & IC Logic Families

Quine-McCluskey Prime Implicant Chart

Page 7

Concept: Setting up rows as Prime Implicants and columns as Minterms, finding essential columns with single X.

> Prime Implicant Table Selection : -

- Columns = all original minterms.
- Rows = all unchecked Prime Implicants.
- Place X in row i, column j if Prime Implicant i covers minterm j.
- Identify columns with only a single X -> corresponding row is an Essential Prime Implicant (EPI).

Complete Tabular Method Worked Example

Concept: Full minimization of $f(w,x,y,z) = \sum m(0,2,3,6,7,8,10,12,13)$ resulting in $f =$

> Final Reduction Verification :-

- Prime Implicants: $(12,13) = wx \bar{y}$; $(0,2,8,10) = x \bar{z}$; $(2,3,6,7) = w \bar{y}$.
- Minimal expression: $f = wx \bar{y} + w \bar{y} + x \bar{z}$.

Digital IC Gates & Characteristics (Delay, Power, Noise Margin)

Concept: Standard 74xx IC gates, Propagation delay (t_p), Power dissipation (PD), and Noise Margin.

> Standard TTL 74xx IC Part Numbers : -

IC Number	Description
7400	Quad 2-input NAND gates
7402	Quad 2-input NOR gates
7404	Hex Inverters (NOT)
7408	Quad 2-input AND gates
7432	Quad 2-input OR gates
7486	Quad 2-input Ex-OR gates

> Key IC Performance Metrics : -

- Propagation Delay (t_p): Time interval between application of input pulse and occurrence of resulting output pulse. Shorter delay = higher operational speed.
- Power Dissipation (PD): Amount of power dissipated: $PD = I_{cc(avg)} \times V_{cc}$.
- Noise Margin: Voltage limit of noise that can be present without disturbing normal circuit operation.

Fan-In, Fan-Out, SPP & Logic Voltage Levels

Concept: Fan-in, Fan-out, Speed Power Product, V_{IH} , V_{IL} , V_{OH} , V_{OL} , and high/low noise margin equations.

> Definitions :-

- Fan-In: Maximum number of inputs that can be connected to a logic gate without disturbing normal operation.
- Fan-Out: Maximum number of standard logic inputs the gate output can reliably drive without exceeding specified voltage limits.
- Speed Power Product (SPP): $SPP = \text{Propagation delay (sec)} \times \text{Power dissipation (W)}$ in Joules. Lower SPP = superior figure of merit.

> Noise Margin Equations :-

Eq: High Level Noise Margin: $V_{NH} = V_{OH}(\min) - V_{IH}(\min)$

Eq: Low Level Noise Margin: $V_{NL} = V_{IL}(\max) - V_{OL}(\max)$

Concept: Comprehensive tree: Bipolar (RTL, DTL, TTL, ECL, I²L) vs Unipolar MOS (NMOS, PMOS, CMOS).

> Logic Families Hierarchy : -

- Bipolar Saturated: RTL (Resistor-Transistor Logic), DTL (Diode-Transistor Logic), TTL (Transistor-Transistor Logic), I²L (Integrated Injection Logic), HTL (High Threshold Logic).
- Bipolar Unsaturated: Schottky TTL, ECL (Emitter Coupled Logic - fastest logic family).
- Unipolar / MOS: PMOS, NMOS, CMOS (Complementary MOS - lowest power dissipation).

TTL Two-Input NAND Gate Circuit & Totem-Pole Output

Concept: Detailed transistor-level schematic of standard TTL NAND gate with multi-emitter transistor Q1 and totem pole (Q3, Q4, D1).

> **Circuit Components :-**

- Q1: Multi-emitter input transistor (acts as AND diodes).
- Q2: Phase splitter transistor providing complementary drive signals to totem pole.
- Totem-Pole Output: Q3 (pull-up transistor), Q4 (pull-down transistor), D1 (diode ensuring Q3 is off when Q4 is on).

CMOS Inverter (NOT Gate) Circuit & Operation

Concept: Complementary PMOS pull-up and NMOS pull-down transistors, switch model, and transfer behavior.

> CMOS Inverter Structure : -

- PMOS connected to VDD (+5V), NMOS connected to Ground (0V). Both gates tied to common input.
- Input = 0V (LOW): PMOS is ON (closed switch), NMOS is OFF (open switch) -> Output pulled up to VDD = +5V.
- Input = +5V (HIGH): PMOS is OFF, NMOS is ON -> Output pulled down to Ground = 0V.
- No direct path from VDD to Ground in steady state -> Static power dissipation is near ZERO!

CMOS NAND Gate Schematic & Operation Cases

Concept: Two parallel PMOS transistors and two series NMOS transistors forming 2-input CMOS NAND.

> **Architecture Rule for CMOS :-**

- NAND: PMOS in PARALLEL, NMOS in SERIES.
- If any input is 0, corresponding PMOS conducts, pulling output to $V_{DD} = 1$.
- Output goes to 0 ONLY when both inputs are 1 (both series NMOS transistors conduct to ground).

CMOS NOR Gate Schematic & Operation Cases

Concept: Two series PMOS transistors and two parallel NMOS transistors forming 2-input CMOS NOR.

> CMOS NOR Architecture : -

- NOR: PMOS in SERIES, NMOS in PARALLEL.
- Output is HIGH (1) ONLY when both inputs are 0 (both series PMOS transistors are ON).
- If either input is 1, parallel NMOS pulls output to ground (0).

Comparison Table: TTL vs CMOS Logic Families

Concept: Definitive 13-parameter side-by-side comparison table between TTL and CMOS.

> TTL vs CMOS Comparison : -

Parameter	CMOS	TTL
1. Devices used	MOSFET (P & N-channel)	BJT (NPN bipolar)
2. VIH (min)	3.5 V	2.0 V
3. VIL (max)	1.5 V	0.8 V
4. VOH (min)	4.95 V	2.7 V
5. VOL (max)	0.05 V	0.4 V
6. VNH (high noise margin)	1.45 V	0.7 V
7. VNL (low noise margin)	1.45 V	0.4 V
8. Noise immunity	Better than TTL	Less than CMOS
9. Propagation delay	70 nsec	10 nsec
10. Power dissipation	0.1 mW (near 0 static)	10 mW
11. Fan-out	50	10
12. Power supply range	3 to 15 V (wide)	Fixed 5 V (strict $\pm 5\%$)
13. Switching speed	Slower than TTL	Faster than CMOS

Concept: Voltage incompatibility analysis (TTL $V_{OH} = 2.7V < CMOS V_{IH} = 3.5V$) and pull-up resistor solution.

> The Interfacing Problem :-

- TTL minimum HIGH output $V_{OH}(\min) = 2.7V$.
- CMOS minimum HIGH input $V_{IH}(\min) = 3.5V$.
- Since $2.7V < 3.5V$, standard TTL cannot reliably drive CMOS directly in HIGH state!

> Solution :-

- Connect external pull-up resistor R_p (typically $1\text{ k}\Omega$ to $10\text{ k}\Omega$) from TTL output to +5V rail.
- When TTL output is HIGH, pull-up resistor pulls voltage up to full +5V rail, safely exceeding CMOS V_{IH} requirement.

IC Interfacing: CMOS Driving TTL & Tri-State Logic

Concept: Current sinking limits, CMOS buffer 4050B, and Tri-state logic (Low, High, High-Impedance).

> CMOS Driving TTL :-

- Voltage is fully compatible (CMOS $V_{OH} = 4.95V > TTL V_{IH} = 2.0V$).
- Current sinking issue: TTL requires 1.6 mA at LOW, while standard CMOS provides only 0.4 mA.
- Solution: Use CMOS buffer IC 4050B / 4049B to boost drive current.

> Tri-State Logic :-

- Outputs have three valid states: LOW (0), HIGH (1), and HIGH IMPEDANCE (Hi-Z).
- In Hi-Z state, output behaves as an open switch (floating), allowing multiple circuit outputs to connect to a shared data bus.

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Combinational vs Sequential Circuits

Page 1

Concept: Core comparison between combinational logic and sequential circuits with memory elements.

> Fundamental Differences : -

Combinational Circuit	Sequential Circuit
Present output depends ONLY on present inputs	Present output depends on present inputs AND past history (previous output)
No memory element required	Memory element (latch/flip-flop) is required
Faster in speed (no feedback delay)	Slower than combinational due to propagation delays
Easy to design	Comparatively harder to design
Examples: Adders, MUX, Demux, Encoders	Examples: Counters, Shift Registers, Serial Adders

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Clock Signals & Latches vs Flip-Flops

Page 2

Concept: Level-triggered vs edge-triggered clocks, 1-bit memory cell, and latch vs flip-flop distinction.

> **Clock Pulses : -**

- Level-triggered clock: Circuit responds as long as clock is active HIGH (+ve level) or active LOW (-ve level).
- Edge-triggered clock: Circuit transitions ONLY during the instantaneous transition from 0 to 1 (+ve edge) or 1 to 0 (-ve edge).

> **Latch vs Flip-Flop : -**

- Latch: Controlled by clock LEVEL. Transparent to inputs while enable signal is active.
- Flip-Flop: Controlled by clock EDGE. Samples inputs and updates output strictly at the triggering edge.

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Basic SR Latch using NOR Gates

Page 3

Concept: Cross-coupled NOR gate latch, truth table, and derivation of Set, Reset, No-change, and Invalid states.

> Truth Table & Operation :-

- $S=1, R=1$ produces $Q=0$ and $\bar{Q}=0$ simultaneously, violating the complement

S	R	Q(t+1)	Q (t + 1)	State of Operation
0	0	Q(t)	Q (t)	No Change (Memory)
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Invalid / Forbidden State

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Basic SR Latch using NAND Gates

Page 4

Concept: Active-low cross-coupled NAND latch schematic, truth table, and operating conditions.

> **NAND Latch Truth Table : -**

S	R	Q(t+1)	Q (t + 1)	State of Operation
0	0	1	1	Indeterminate / Forbidden
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q(t)	Q (t)	No Change (Memory)

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Clocked SR Flip-Flop

Page 5

Concept: Clock gating using steering NAND gates, timing behavior, and characteristic derivation.

> Clocked SR Flip-Flop Truth Table :-

Clock	S	R	Q(t+1)	State
0	X	X	Q(t)	No Change
!'	0	0	Q(t)	No Change
!'	0	1	0	Reset
!'	1	0	1	Set
!'	1	1	?	Undetermined / Forbidden

* Unit 3: Sequential Circuits, Flip-Flops & Counters

SR Characteristic Equation & Excitation Table

Page 7

Concept: K-map derivation of $Q(t+1) = S + R \cdot Q$ and excitation table required for counter

> Characteristic Equation :-

$$Eq : Q(t+1) = S + R \cdot Q \quad (\text{with condition } S \cdot R = 0)$$

> SR Flip-Flop Excitation Table :-

Q(t)	Q(t+1)	S	R
0	0	0	X (don't care)
0	1	1	0
1	0	0	1
1	1	X	0

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Clocked D Flip-Flop (Delay / Data)

Page 8

Concept: Single data line with inverter to eliminate forbidden state, characteristic equation $Q(t+1) = D$.

> D Flip-Flop Equations & Excitation :-

$$\text{Eq: } Q(t+1) = D$$

$$\text{Eq: } D = Q(t+1)$$

Q(t)	Q(t+1)	D (Excitation)
0	0	0
0	1	1
1	0	0
1	1	1

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Clocked JK Flip-Flop (Jack Kilby)

Page 9

Concept: Cross-coupled feedback from outputs to inputs eliminating the forbidden state and enabling toggle.

> JK Truth Table :-

Clock	J	K	Q(t+1)	State of Operation
0	X	X	Q(t)	No Change
1'	0	0	Q(t)	No Change
1'	0	1	0	Reset
1'	1	0	1	Set
1'	1	1	Q (t)	Toggle (Complements State)

* Unit 3: Sequential Circuits, Flip-Flops & Counters

JK Characteristic Equation & Excitation Table

Page 10

Concept: K-map derivation of $Q(t+1) = J \cdot Q + K \cdot Q$ and JK excitation table.

> Characteristic Equation :-

$$Eq : Q(t+1) = J \cdot Q(t) + K \cdot Q(t)$$

> JK Excitation Table :-

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Clocked T Flip-Flop (Toggle Flip-Flop)

Page 11

Concept: Formed by tying J and K inputs together; toggles output when T=1.

> **T Flip-Flop Equations :-**

$$Q(t+1) = T \cdot Q(t) + \overline{T} \cdot \overline{Q(t)}$$
$$T = Q(t) \oplus Q(t+1)$$

Q(t)	Q(t+1)	T (Excitation)
0	0	0
0	1	1
1	0	1
1	1	0

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Asynchronous (Ripple) vs Synchronous Counters

Page 15

Concept: Ripple clock chaining versus simultaneous common clock distribution.

> Counter Classification :-

- Asynchronous (Ripple) Counter: Clock signal is applied ONLY to first flip-flop. Subsequent flip-flops are clocked by output of preceding stage.
- Synchronous Counter: Common clock is applied SIMULTANEOUSLY to all flip-flops. State transitions happen in lockstep.
- MOD-N Counter: Counts N distinct states. Number of flip-flops n satisfies:

MOD-10 (Decade / BCD) Ripple Counter

Concept: 4-stage JK ripple counter with NAND reset gate triggering on count 10 ($1010_2 = Q_D Q_C$).

> Design Details :-

- Total counts: 0 to 9 (10 states). Requires $n=4$ flip-flops ($2^4 = 16 > 10$).
- At the 10th count (1010₂), $Q_D=1$ and $Q_C=1$.
- Feed Q_D and Q_C into a 2-input NAND gate connected to the active-low CLEAR inputs of all flip-flops.
- Instantaneous reset occurs, forcing counter back to 0000.

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Shift Registers: SISO, SIPO, PIPO, PISO Architecture

Page 23

Concept: 4-bit bidirectional shift register modes, serial input/output timing waveforms.

> **Shift Register Modes : -**

- SISO (Serial In Serial Out): Requires n clock cycles to load and $n-1$ cycles to retrieve.
- SIPO (Serial In Parallel Out): Serial loading in n clock cycles, all bits available simultaneously.
- PIPO (Parallel In Parallel Out): Instantaneous 1-clock parallel load and parallel read.
- PISO (Parallel In Serial Out): Uses multiplexing steer logic for parallel load / serial shift.

* Unit 3: Sequential Circuits, Flip-Flops & Counters

Ring Counter & Johnson (Twisted Ring) Counter

Page 26

Concept: Circulating single 1 counter (MOD-N) vs inverted feedback Johnson counter (MOD-2N).

> **Ring vs Johnson Counter** : -

Feature	Ring Counter	Johnson (Twisted Ring) Counter
Feedback	Output Q of last stage to D of first stage	Inverted output Q of last stage to D of first stage
Total States for n FFs	n states (e.g. 4 states for 4 FFs)	2n states (e.g. 8 states for 4 FFs)
State Sequence (4 FFs)	1000 -> 0100 -> 0010 -> 0001	0000 -> 1000 -> 1100 -> 1110 -> 1111 -> 0111 -> 0011 -> 0001
Self-Starting Requirement	Requires preset/clear initialization	Easier self-recovery with feedback decoding

* Unit 4: Combinational Logic Circuits

Half Adder Circuit & Implementation

Page 1

Concept: Two-operand addition, truth table, $Sum = A \oplus B$, $Carry = A \cdot B$, and XOR/AND

> Equations & Truth Table :-

$$Eq : Sum = A \oplus B + A \cdot B = A \oplus B$$

$$Eq: Carry = A \cdot B$$

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

* Unit 4: Combinational Logic Circuits

Full Adder Circuit & K-Map Derivation

Page 2

Concept: Three-input addition (A, B, Cin) with K-map simplification for Sum and Carry.

> **Full Adder Equations :-**

$$\text{Eq : Sum} = A \oplus B \oplus C_{in}$$

$$\text{Eq : Carry (Cout)} = AB + BC_{in} + AC_{in} = AB + C_{in}(A + B)$$

Look-Ahead Carry Adder (CLA) Principles

Concept: Eliminating ripple carry propagation delay using Carry Generate (Gi) and Carry Propagate (Pi).

> Carry Generate & Propagate Equations :-

- All carries are generated simultaneously in 2 gate levels directly from inputs.
- Eliminates ripple delay: addition time is independent of the number of bits.

Eq: Generate: $G_i = A_i \cdot B_i$

Eq: Propagate: $P_i = A_i \oplus B_i$

Eq: $C_1 = G_0 + P_0 \cdot C_0$

Eq: $C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$

Eq: $C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$

Eq: $C_n = G_n + P_n \cdot G_{n-1} + P_n \cdot P_{n-1} \cdot G_{n-2} + \dots + P_n \cdot P_{n-1} \cdot P_{n-2} \cdot \dots \cdot P_0 \cdot C_0$

BCD Addition Rules & +6 Correction Factor

Concept: Handling invalid sums > 9 or carry generated by adding 0110 (6) to the nibble.

> BCD Addition Algorithm :-

- 1. Add two 4-bit BCD numbers using binary addition.
- 2. If the 4-bit sum is "d 9 (1001) and Cout = 0, the sum is a valid BCD number.
- 3. If the sum > 9 OR a carry Cout = 1 is generated, the result is INVALID BCD.
- 4. To correct: ADD 0110, (6•€) to the 4-bit sum group.

Q) Add (45)•€ + (26)•€ in BCD

- > Group 1 (units): $0101 + 0110 = 1011$ (11•€ > 9 -> INVALID).
- > Add 0110: $1011 + 0110 = (1)0001$ -> Sum digit = 1, Carry = 1.
- > Group 2 (tens): $0100 + 0010 + 1(\text{carry}) = 0111$ (7•€ "d 9 -> VALID).
- > Final BCD: $0111\ 0001 = (71)•€$.

* Unit 4: Combinational Logic Circuits

Magnitude Comparators (1-bit & 2-bit)

Page 13

Concept: Hardware comparison for $A > B$, $A < B$, and $A = B$ using XOR, XNOR, and AND gates.

> 1-Bit Comparator Equations :-

$$Eq : A > B \text{ (Greater)} : G = A \cdot \overline{B}$$

$$Eq : A < B \text{ (Lesser)} : L = \overline{A} \cdot B$$

$$Eq : A = B \text{ (Equal)} : E = A \oplus B = \overline{A \cdot B} + A \cdot B$$

> 2-Bit Comparator Logic :-

$$Eq : E = (\overline{A_1} \cdot \overline{B_1}) \cdot (A_0 \oplus B_0)$$

$$Eq : G = A_1 \cdot B_0 + (\overline{A_1} \cdot \overline{B_1}) \cdot A_0 \cdot B_0$$

$$Eq : L = \overline{A_1} \cdot B_1 + (\overline{A_1} \cdot \overline{B_1}) \cdot \overline{A_0} \cdot B_0$$

Multiplexers (2x1, 4x1, 8x1 MUX Architecture)

Concept: Data selectors with 2^n inputs, n select lines, and 1 output.

> **General Formula :-**

$$\text{Eq : } 2 \times 1 \text{ MUX : } Y = S_1 \cdot I_0 + S_0 \cdot I_1$$

$$\text{Eq : } 4 \times 1 \text{ MUX : } Y = S_2 \cdot S_1 \cdot I_0 + S_2 \cdot S_0 \cdot I_1 + S_1 \cdot S_0 \cdot I_2 + S_2 \cdot S_1 \cdot I_3$$

* Unit 4: Combinational Logic Circuits

Decoders (3 to 8 Line Decoder IC 74138)

Page 30

Concept: Active-high and active-low decoders with enable pins, generating all 2^n minterms.

> **Equations :-**

$$E q : Y_0 = E \cdot A \cdot B \cdot C$$

$$E q : Y_1 = E \cdot A \cdot B \cdot \bar{C}$$

Eq: ...

$$E q : Y_7 = E \cdot A \cdot \bar{B} \cdot \bar{C}$$

* Unit 4: Combinational Logic Circuits

Priority Encoders (4 to 2 Line)

Page 34

Concept: Resolving simultaneous inputs where highest index input takes priority, using don't cares.

> **Priority Encoder Equations :-**

$$\begin{aligned} E_q : A &= D_1 + D_2 f \\ E_q : B &= D_1 f + D_2, \cdot D \cdot \end{aligned}$$

Memory Organization, Capacity & Block Diagram

Concept: K address lines for 2^K memory words, n-bit word length, memory capacity, read/write timing.

> Core Memory Principles : -

- 1-bit memory cell stores 1 bit (0 or 1).
- Registers are made of flip-flops to store word data.
- Memory is an array of registers, each identified by a unique address.
- Capacity = $2^K \times n$ bits (K address lines, n data lines). Example: 10 address words.

* Unit 5: Semiconductor Memories & PLDs

Classification of Semiconductor Memories

Page 2

Concept: Hierarchy: Volatile (RAM - SRAM, DRAM) vs Non-Volatile (ROM - Masked, PROM, EPROM, EEPROM, Flash).

> **Memory Classification Tree : -**

- RAM (Random Access / Read-Write, Volatile): Loses data when power is off. Subdivided into Static RAM (SRAM) and Dynamic RAM (DRAM).
- ROM (Read Only Memory, Non-Volatile): Retains data permanently even when power is removed.
- CAM (Content Addressable Memory): Accesses data by value rather than address.

Comparison: Static RAM (SRAM) vs Dynamic RAM (DRAM)

Concept: Definitive side-by-side comparison table between 6T SRAM and 1T-1C DRAM.

> SRAM vs DRAM Comparison Table : -

Parameter	SRAM (Static RAM)	DRAM (Dynamic RAM)
1. Storage Element	Cross-coupled Flip-Flops (4 to 6 transistors)	MOSFET + Storage Capacitor (1T-1C cell)
2. Refreshing	NO refreshing required	Requires continuous periodic refreshing (every 2-4 ms)
3. Speed / Access Time	Faster (10 to 30 ns access time)	Slower (50 to 100 ns access time)
4. Packing Density	Low density (fewer bits per unit chip area)	High density (millions of bits per chip)
5. Power Consumption	Higher power consumption	Lower power consumption
6. Cost	More expensive per bit	Significantly cheaper per bit
7. Application	CPU Cache memory (L1, L2, L3)	Main computer system memory (DIMMs)

ROM Types: PROM, EPROM, EEPROM & Flash Memory

Concept: Comparison of programming and erasing methods across non-volatile semiconductor memory technologies.

> ROM Family Comparison : -

Type	Programming Method	Erasing Method	Re-writability
Masked ROM	Hardwired during fabrication	Cannot be erased	No (read only)
PROM	Electrically by burning fusible links	Cannot be erased	Once only (OTP)
EPROM	Hot electron injection by programming	UV light exposure through quartz window	Multiple times (entire chip erased)
EEPROM	Fowler-Nordheim electrical tunneling	Electrically byte-by-byte in circuit	100,000+ cycles
Flash Memory	Electrical Fowler-Nordheim tunneling	Electrically block-by-block (sector)	Fast & high density

Concept: Programmable Logic Devices comparison: fixed vs programmable AND and OR array matrices.

> Three Major Types of Combinational PLDs :-

- 1. PROM: Fixed AND array (decoder) + Programmable OR array.
- 2. PAL (Programmable Array Logic): Programmable AND array + Fixed OR array (easier to program, highly popular).
- 3. PLA (Programmable Logic Array): Programmable AND array + Programmable OR array (most flexible, both arrays programmable).

PLD Comparison Table & Realization Examples

Concept: Comprehensive comparison table and worked realization examples for Full Adder using PLA and PAL.

> Architectural Comparison :-

Feature	PROM	PLA	PAL
AND Array	Fixed (decoder)	Programmable	Programmable
OR Array	Programmable	Programmable	Fixed
Cost & Complexity	Cheapest & simplest	Costliest and most complex	Moderate cost, simpler than PLA
Minterms generated	All 2^n minterms decoder	Only required product terms	Only required product terms
Flexibility	Standard SOP only	Highest flexibility	High speed, fixed OR connections